Homework #1

Vanja Stojanović

March 2026

1 Connected components

We assume a graph G s.t. $|V(G)| = n$ and $|E(G)| = m$, and it contains c connected components. We prove the following:

Proposition 1.

$$n - c \leq m \leq \binom{n-c+1}{2}$$

Proof. We proceed with induction on n for the first inequality.
Base case (n = 1). It trivially follows that $c = 1$ and $m = 0$, hence vacuously $1 - 1 = 0 \leq 0$.
Induction hypothesis (IH). We assume the relation holds for n.
Induction step. Assume for a graph G with $V(G) = n$.

Let G' be the graph we obtain by adding a vertex v to G. By the IH our inequality holds for G. When we add a vertex v to G, we can arbitrarily choose its degree in the interval $0 \leq \deg(v) \leq n$, hence we increase the number of vertices to $n' = n + 1$, the number of connected components can stay the same or increase by one if $\deg(v) = 0$ and we increase the number of edges to $m' = m + \deg(v)$.
If $\deg(v) \geq 1$, then c' in G' can only decrease or stay the same. If v connects to k distinct components (where $k \leq \deg(v)$), then $c' = c - k + 1$:

$$(n+1) - (c-k+1) \leq m + \deg(v) \tag{1}$$
$$n - c + k \leq m + \deg(v) \tag{2}$$

which holds by the IH and the fact that $k \leq \deg(v)$.
If $\deg(v) = 0$, then

$$(n+1) - (c+1) \leq m + 0 \tag{3}$$
$$n - c \leq m \tag{4}$$

which again holds by the IH, proving our induction step.

For the second inequality, let n_i denote the number of vertices in component i. Then $n = \sum_{i=1}^{c} n_i$ with $n_i \geq 1$, and each component has at most $\binom{n_i}{2}$ edges, so

$$m \leq \sum_{i=1}^{c} \binom{n_i}{2}.$$

We claim this sum is maximized when one component has $n-c+1$ vertices and the remaining $c-1$ are singletons. To see this, suppose two components have $n_a \geq 2$ and $n_b \geq 2$. Moving a vertex from b to a changes the sum by

$$\binom{n_a+1}{2} + \binom{n_b-1}{2} - \binom{n_a}{2} - \binom{n_b}{2} = n_a - (n_b - 1) = n_a - n_b + 1.$$

If $n_a \geq n_b$, this is positive, so the sum strictly increases. Repeating this process, we concentrate all non-singleton vertices into one component, giving

$$m \leq \binom{n-c+1}{2}.$$

□

By Harary's theorem [1], a disconnected graph on n vertices has at most $\binom{n-1}{2}$ edges. This follows directly from our upper bound with $c \geq 2$:

Corollary 2. If $m > \binom{n-1}{2}$, then G is connected.

Proof. By contrapositive. If G is disconnected ($c \geq 2$), then by Proposition 1, $m \leq \binom{n-c+1}{2} \leq \binom{n-1}{2}$. □

Remark 3. This criterion is not practically useful for real networks. Most real-world graphs, like social networks [2], internet topology, and biological networks are sparse, with $m = O(n)$ or $O(n \log n)$, yet are typically connected. The threshold $\binom{n-1}{2} = O(n^2)$ is far too conservative. For instance, Erdős and Rényi [4] showed that the sharp threshold for connectivity in the random graph $G(n,p)$ is $p = \frac{\ln n}{n}$, corresponding to $m \approx \frac{n \ln n}{2}$, which is much smaller than $O(n^2)$.

2 Node linking probability

We consider such a random graph G, where an edge between $i \in V(G)$ and $j \in V(G)$ is present with probability p_{ij}. The probability is proportional to $p_{ij}\ v_i v_j$, where $v_i \geq 0$ is some number associated with node i.

Proposition 4. The expected node degree $\mathbb{E}(k_i)$ of a node i is proportional to v_i, where $k_i = deg(i)$.

Proof. Since each edge is a Bernoulli trial, we can express the expected value of the node degree of $i \in V(G)$ as

$$\mathbb{E}(k_i) = \sum_{j \neq i} p_{ij} = \sum_{j \neq i} \alpha v_i v_j = \alpha v_i \sum_{j \neq i} v_j$$

hence $\mathbb{E}(k_i) = C v_i$, where $C = \alpha \sum_{j \neq i} v_j$ and α represents the unknown proportionality constant. □

We can now express p_{ij} in terms of the expected node degree.

Proposition 5. We can write the probability p_{ij} in terms of the expected vertex degree of vertices i and j.

Proof. By Proposition 4 we have $\mathbb{E}(k_i) = v_i\big(\alpha \sum_{j \neq i} v_j\big)$. We substitute it into the relation for p_{ij}

$$\begin{aligned}
p_{ij} &= \alpha \cdot \frac{\mathbb{E}(k_i)}{\alpha \sum_j v_j} \cdot \frac{\mathbb{E}(k_j)}{\alpha \sum_j v_j} \\
&= \frac{\mathbb{E}(k_i)\mathbb{E}(k_j)}{\alpha \big(\sum_j v_j\big)^2} \\
&= \frac{\mathbb{E}(k_i)\mathbb{E}(k_j)}{\alpha \Big(\sum_j v_j\Big)\Big(\sum_l v_l\Big)} \\
&= \frac{\mathbb{E}(k_i)\mathbb{E}(k_j)}{\sum_l \alpha v_l \Big(\sum_j v_j\Big)} \\
&= \frac{\mathbb{E}(k_i)\mathbb{E}(k_j)}{\sum_l \mathbb{E}(k_l)}
\end{aligned}$$

where we use Proposition 4 in the last step. □

The derivation in the proof of Proposition 5 is formally known as the Chung-Lu model [3].

3 Random node selection

Assume for a graph G. We design an algorithm which takes in a graph (network) data structure representation, and chooses a random node (vertex) in $O(1)$, where the probability of a node being chosen is $p = \frac{deg(i)}{2m}$ where $i \in V(G)$.

Algorithm 1 Degree-proportional node selection

Require: Edge list $E = [e_1, e_2, \ldots, e_{2m}]$ containing both endpoints of each edge

```
idx ← UniformRandom(1, 2m)
return E[idx]
```

Theorem 6. Algorithm 3 selects a node with probability $P(i) = \frac{deg(i)}{2m}$.

Proof. Assume E is the edge list inputted into the algorithm. The edge list is constructed out of both endpoints of all edges, since each edge has 2 endpoints, the number of elements in the list is $2m$. Hence, when picking a node uniformly at random from the list, we can write the following equation

$$P(i) = \frac{|\{x : x \in E \wedge x = i\}|}{2m}$$

since each vertex will appear exactly as many times in the list as is the number of its edges

$$P(i) = \frac{deg(i)}{2m}$$

□

4 Weak & strong connectivity

For this section, we assume for a digraph D. If we were to apply the depth-first search (DFS hereon) to D and take into account the three given criteria, the result would be:

1. **If we follow the links as if the ordered pair would be unordered**. Then the algorithm would be equivalent as if we took the unoriented graph G of D and applied the DFS on it. Hence we would get weakly connected components of D.

2. **If we follow the links in the direction of the ordered pair**. We get the "visibility" graph of some vertex, i.e. which vertices we can reach starting at some vertex.

3. **If we follow the links in the opposite direction of the ordered pair**. We get the "reverse-visibility" graph of some vertex, i.e. which vertices can reach that vertex.

From answers 2 and 3, vertex u is in the same SCC as v iff u is reachable from v following directed edges, and v is reachable from u. We use this to find all SCCs:

Algorithm 2 Strongly connected components via forward and backward DFS

Require: Digraph $D = (V, E)$
Ensure: Set of all SCCs

```
unvisited ← V
SCCs ← ∅
while unvisited ≠ ∅ do
    Pick any v ∈ unvisited
    F ← DFS(v, D)                    ▷ vertices reachable from v
    B ← DFS(v, D^R)                  ▷ vertices that can reach v
    SCC(v) ← F ∩ B
    SCCs ← SCCs ∪ {SCC(v)}
    unvisited ← unvisited \ SCC(v)
end while
return SCCs
```

Implementing the algorithm and running it on the `enron.net` network, we get that the network has 78,058 strongly connected components, and the larget one has 9,164 of vertices. This is expected for a communication network like Enron's email graph. Most vertices are in small SCCs (many singletons — people who only sent or only received emails), while there's one large SCC forming the core of bidirectional communication.

5 Is Java software scale-free?

We analyze the Lucene search engine class dependency network [5] with $n = 2956$ vertices and $m = 10872$ arcs. Figure 1 shows the degree distributions on a log-log plot.
Using the MLE formula from [6] with $k^{\cdot}_{\min} = 5$, computed separately for in- and out-degree:

$$\gamma_{\cdot} = 1 + n_{\cdot} \left(\sum_{i=1}^{n} \ln \frac{k^{\cdot}_i}{k^{\cdot}_{\min} - \frac{1}{2}} \cdot \delta(k^{\cdot}_i \geq k^{\cdot}_{\min}) \right)^{-1}$$

where $\delta(k^{\cdot}_i \geq k^{\cdot}_{\min})$ is the indicator function and $n_{\cdot} \leq n$ is the number of nodes with $k^{\cdot}_i \geq k^{\cdot}_{\min}$. We obtain:

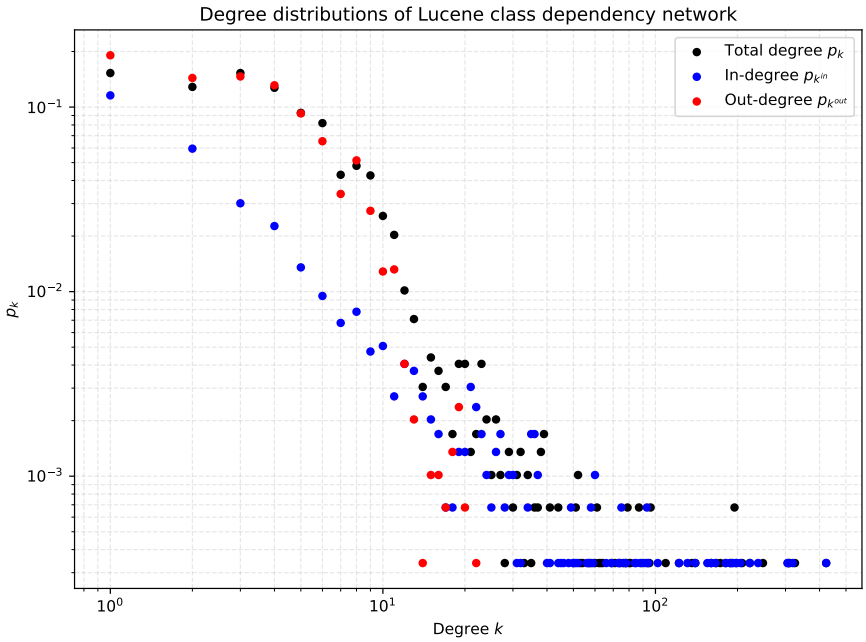

Figure 1: Degree distributions of the Lucene class dependency network.

- **In-degree**: appears scale-free, roughly linear on the log-log plot. $\gamma_{\text{in}} \approx 1.80$ ($n_{\text{in}} = 315$). This is consistent with preferential attachment [2]: widely-used utility classes (interfaces, base classes) accumulate dependencies from many other classes.

- **Out-degree**: appears scale-free, though with more curvature than in-degree. $\gamma_{\text{out}} \approx 3.37$ ($n_{\text{out}} = 917$). The steeper exponent reflects that out-degree (how many dependencies a class has) is more constrained by software design principles such as limited coupling.

- **Total degree**: appears scale-free as a mixture of both distributions. $\gamma_{\text{total}} \approx 2.47$ ($n_{\text{total}} = 1296$), sitting between in- and out-degree on the log-log plot.

The code used to produce the plot and compute the exponents:

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict, Counter


def parse_net(path):
    in_degree = defaultdict(int)
    out_degree = defaultdict(int)
```

```python
    vertices = set()

    with open(path) as f:
        section = None
        for line in f:
            line = line.strip()
            if line.startswith("*vertices"):
                section = "vertices"
                continue
            elif line.startswith("*arcs"):
                section = "arcs"
                continue

            if section == "vertices":
                vid = int(line.split()[0])
                vertices.add(vid)
                in_degree[vid] = in_degree.get(vid, 0)
                out_degree[vid] = out_degree.get(vid, 0)
            elif section == "arcs":
                parts = line.split()
                u, v = int(parts[0]), int(parts[1])
                out_degree[u] += 1
                in_degree[v] += 1

    return vertices, in_degree, out_degree


def degree_distribution(degree_dict):
    counts = Counter(degree_dict.values())
    n = len(degree_dict)
    ks = sorted(k for k in counts if k > 0)
    pk = {k: counts[k] / n for k in ks}
    return ks, pk


def mle_gamma(degrees, k_min):
    filtered = [k for k in degrees if k >= k_min]
    n = len(filtered)
    if n == 0:
        return None, 0
    gamma = 1 + n * (sum(np.log(k / (k_min - 0.5)) for k in filtered)) ** -1
    return gamma, n


def plot_distributions(total_ks, total_pk, in_ks, in_pk, out_ks, out_pk):
    fig, ax = plt.subplots(figsize=(8, 6))

```

```python
    ax.scatter(
        total_ks,
        [total_pk[k] for k in total_ks],
        c="black",
        s=20,
        label="Total degree $p_k$",
        zorder=3,
    )
    ax.scatter(
        in_ks,
        [in_pk[k] for k in in_ks],
        c="blue",
        s=20,
        label="In-degree $p_{k^{in}}$",
        zorder=3,
    )
    ax.scatter(
        out_ks,
        [out_pk[k] for k in out_ks],
        c="red",
        s=20,
        label="Out-degree $p_{k^{out}}$",
        zorder=3,
    )

    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("Degree $k$")
    ax.set_ylabel("$p_k$")
    ax.set_title("Degree distributions of Lucene class dependency network")
    ax.legend()
    ax.grid(True, which="both", ls="--", alpha=0.3)

    plt.tight_layout()
    plt.savefig("HW1/build/lucene_degree_distributions.pdf")
    plt.savefig("HW1/build/lucene_degree_distributions.png", dpi=150)
    print("Plot saved to HW1/build/lucene_degree_distributions.pdf")


if __name__ == "__main__":
    vertices, in_deg, out_deg = parse_net("HW1/networks/lucene.net")
    total_deg = {v: in_deg[v] + out_deg[v] for v in vertices}

    total_ks, total_pk = degree_distribution(total_deg)
    in_ks, in_pk = degree_distribution(in_deg)
```

```
    out_ks, out_pk = degree_distribution(out_deg)

    plot_distributions(total_ks, total_pk, in_ks, in_pk, out_ks, out_pk)

    print("\nMLE power-law exponents")
    k_min = 5
    for name, deg_dict in [("Total", total_deg), ("In", in_deg), ("Out", out_deg)]:
        degrees = [k for k in deg_dict.values() if k > 0]
        gamma, n_used = mle_gamma(degrees, k_min)
        if gamma:
            print(f"{name}-degree: gamma = {gamma:.3f} (k_min = {k_min}, n = {n_used})")
```

6 Five networks problem

7 Who to vaccinate?

References

[1] F. Harary, *Graph Theory*, Addison-Wesley, 1969.

[2] A.-L. Barabási and R. Albert, Emergence of scaling in random networks, *Science*, 286(5439):509–512, 1999.

[3] F. Chung and L. Lu, Connected components in random graphs with given expected degree sequences, *Annals of Combinatorics*, 6:125–145, 2002.

[4] P. Erdős and A. Rényi, On random graphs I, *Publicationes Mathematicae Debrecen*, 6:290–297, 1959.

[5] L. Šubelj and M. Bajec, Community structure of complex software systems: Analysis and applications, *Physica A*, 390(16):2968–2975, 2011.

[6] A.-L. Barabási, *Network Science*, Cambridge University Press, 2016.